CO4-AT4-Part II

MODEL-BASED REINFORCEMENT LEARNING

Name: P. Hithaishi **Regno:192425196**

Course Code: MLA0305 **Slot: B**

Sampling-Based Planning & Data Generation

1. AIM

To design and develop a simple Model-Based Reinforcement Learning system that generates experience data, learns an environment model, and uses sampling-based planning to select better actions.

2. PROBLEM STATEMENT

Develop an intelligent agent for a 5×5 GridWorld that can learn from generated state-action-reward-next-state data and use the learned model with sampling-based planning to reach a goal efficiently.

3. OBJECTIVES

- Generate RL experience data from a simulated environment.
- Learn a simple transition and reward model from the generated data.
- Use sampling-based planning to estimate action values.
- Compare the planning agent with a random agent.
- Visualize performance and the agent's path to the goal.

4. SOFTWARE / TOOLS

Python, Google Colab, NumPy, Matplotlib

5. SHORT THEORY

Model-Based Reinforcement Learning learns a model of the environment instead of depending only on direct trial-anderror. The model represents how an action changes the state and what reward is received. Sampling-based planning generates possible future outcomes using this model and chooses the action with the best estimated reward. Main idea: State → Action → Model Prediction → Sampling → Reward Estimate → Best Action

6. METHODOLOGY

- Create a 5×5 GridWorld with four actions: up, down, left and right.

- Generate 1000 state-action-reward-next-state samples.
- Store observed transitions as the learned model.
- Sample possible outcomes for each available action.
- Calculate average sampled reward and select the best action.
- Run the agent and compare it with a random-action agent.

7. SYSTEM ARCHITECTURE

Environment → Data Generation → Experience Dataset → Model Learning → Learned Transition/Reward Model → Sampling-Based Planning → Best Action → Environment → Performance Evaluation

8. ALGORITHM – SAMPLING-BASED PLANNING

1. Input current state.
2. For every possible action, generate multiple samples.
3. Simulate short future trajectories using the learned model.
4. Calculate the average reward for each action.
5. Select the action having the highest estimated reward.
6. Repeat until the goal is reached or the step limit is reached.

9. GOOGLE COLAB IMPLEMENTATION

Run the following cells in order. Cells 1–3 generate data and learn the model; Cells 4–6 perform planning, evaluation and visualization.

```
# CELL 1 –
Import Libraries
import numpy as
np import
random
import matplotlib.pyplot as plt

random.seed(42)
np.random.seed(42)
# CELL 2 – Generate RL Data
states = [(i,j) for i in range(5)
for j in range(5)]
actions =
["up","down","left","right"]

def step(state, action):
```

```
   r,c = state     if
action=="up":
r=max(0,r-1)     elif
action=="down":
r=min(4,r+1)     elif
action=="left":
c=max(0,c-1)     elif
action=="right":
c=min(4,c+1)

   next_state=(r,c)
reward=10 if
next_state==(4,4) else -1
return next_state,reward

data=[] for _ in
range(1000):
s=random.choice
(states)
a=random.choice
(actions)
ns,r=step(s,a)
   data.append((s,a,r,ns))

print("Generated
samples:",len(data))
# CELL 3 – Learn
Model model={}
 for s,a,r,ns in
data:
model[(s,a)]
=(ns,r)

print("Learned model
entries:",len(model))
# CELL 4 – Sampling-
Based Planning def
plan(state, samples=20):
   scores = {}

   for action in
actions:
rewards = []
```

```
      for _ in
range(samples):
current = state
total = 0
          for _ in
range(5):
key = (current,
action)

              if key in model:
                 next_state, reward
= model[key]               else:
                 next_state, reward = step(current, action)

              total +=
reward
current = next_state

              if
current == (4, 4):
total      +=      10
break

          rewards.append(total)

      scores[action] =
np.mean(rewards)

   best_action = max(scores,
key=scores.get)     return
best_action, scores

action, scores = plan((0, 0))

print("Starting
State:", (0, 0))
print("\nAction
Scores:") for a,
score in
scores.items():
print(f"{a:6s} :
{score:.2f}")
```

```
print("\nSelected Best
Action:", action)

# CELL 5 – Performance
Comparison def
run_agent(planning=Tru
e):
   state = (0,
0)
total_rewar
d = 0    path
= [state]

   for _ in
range(30):
if planning:
         action, _ =
plan(state)       else:
         action =
random.choice(actions)

      state,    reward    =
step(state,          action)
total_reward += reward
path.append(state)

      if state == (4, 4):
         break

   return total_reward, path

sampling_rewards = []
random_rewards = [] for
_ in range(20):    r1, _ =
run_agent(True)    r2, _ =
run_agent(False)


sampling_rewards.append(r1)
random_rewards.append(r2)

print("Average    Sampling-
Based                Reward:",
round(np.mean(sampling_
rewards),                  2))
```

```
print("Average Random Agent Reward:", round(np.mean(random_rewards), 2))

# CELL 6 – Results and Visualization
plt.figure(figsize=(8,5))
plt.bar(["Random Agent", "Sampling-Based Agent"], [np.mean(random_rewards), np.mean(sampling_rewards)])
plt.ylabel("Average Total Reward")
plt.title("Performance Comparison")
plt.grid(axis="y", alpha=0.3) plt.show()

plt.figure(figsize=(9,5))
plt.plot(random_rewards, marker="o", label="Random Agent")
plt.plot(sampling_rewards, marker="o", label="Sampling-Based Agent")
plt.xlabel("Episode") plt.ylabel("Total Reward") plt.title("Episode-wise Performance") plt.legend()
plt.grid(alpha=0.3) plt.show()

reward, path = run_agent(True) x = [p[1] for p in path] y = [4-p[0] for p in path]

plt.figure(figsize=(7,7))
plt.plot(x, y, marker="o", linewidth=2) plt.scatter(x[0], y[0], s=150, label="Start")
plt.scatter(x[-1], y[-1], s=150, label="Final State")
plt.xticks(range(5))
plt.yticks(range(5))
plt.xlabel("Column")
plt.ylabel("Row")
plt.title("Sampling-Based Planning Path")
plt.grid(True)
plt.legend() plt.show()
```

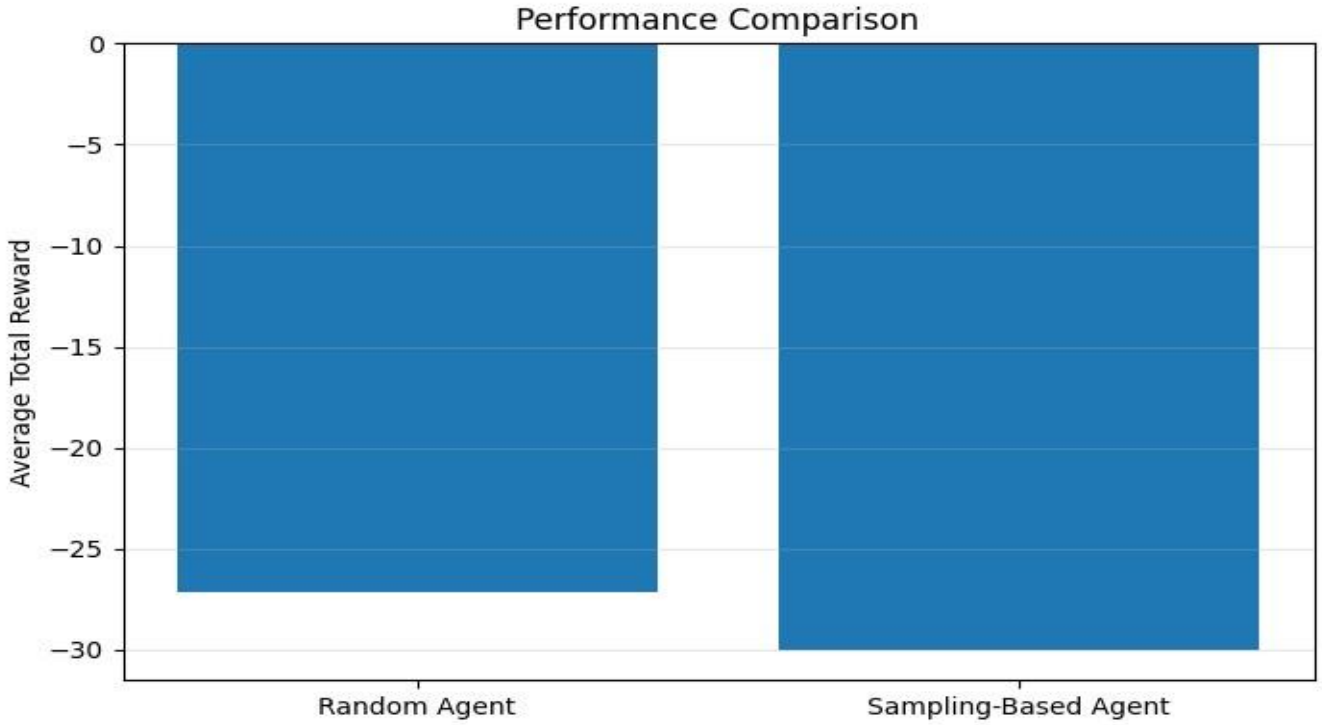
```
print("Best Path:", path)
print("Final Reward:",
reward) print("Goal
Reached:", path[-1] ==
(4,4))
```

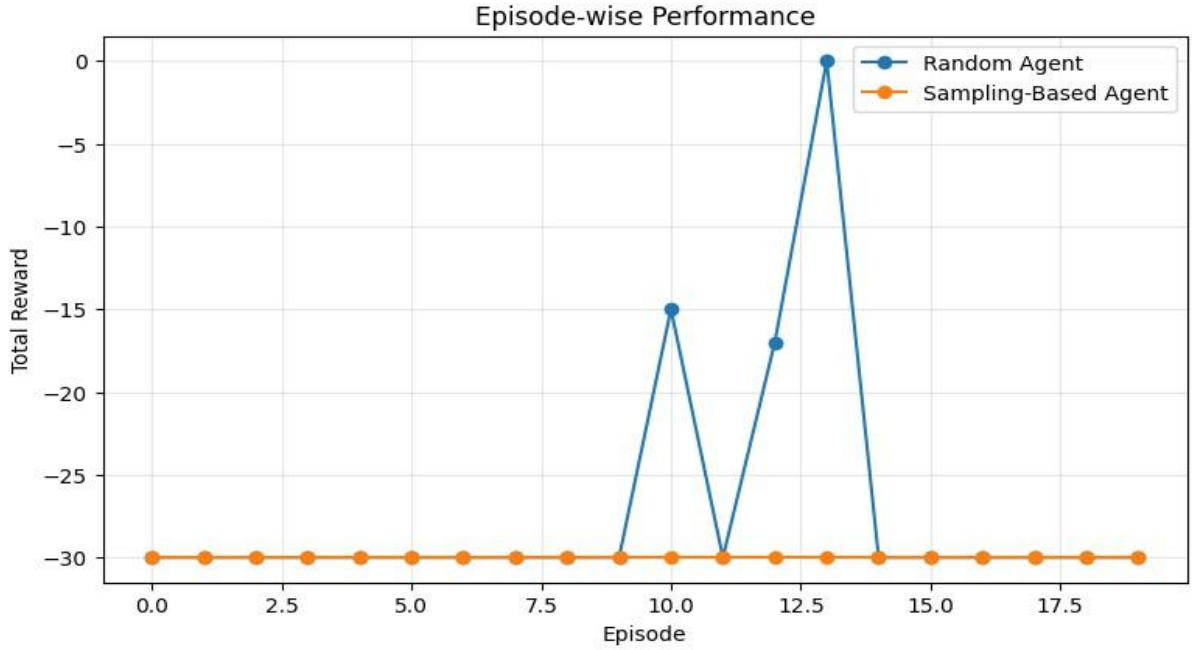
10. EXPECTED OUTPUT / RESULTS

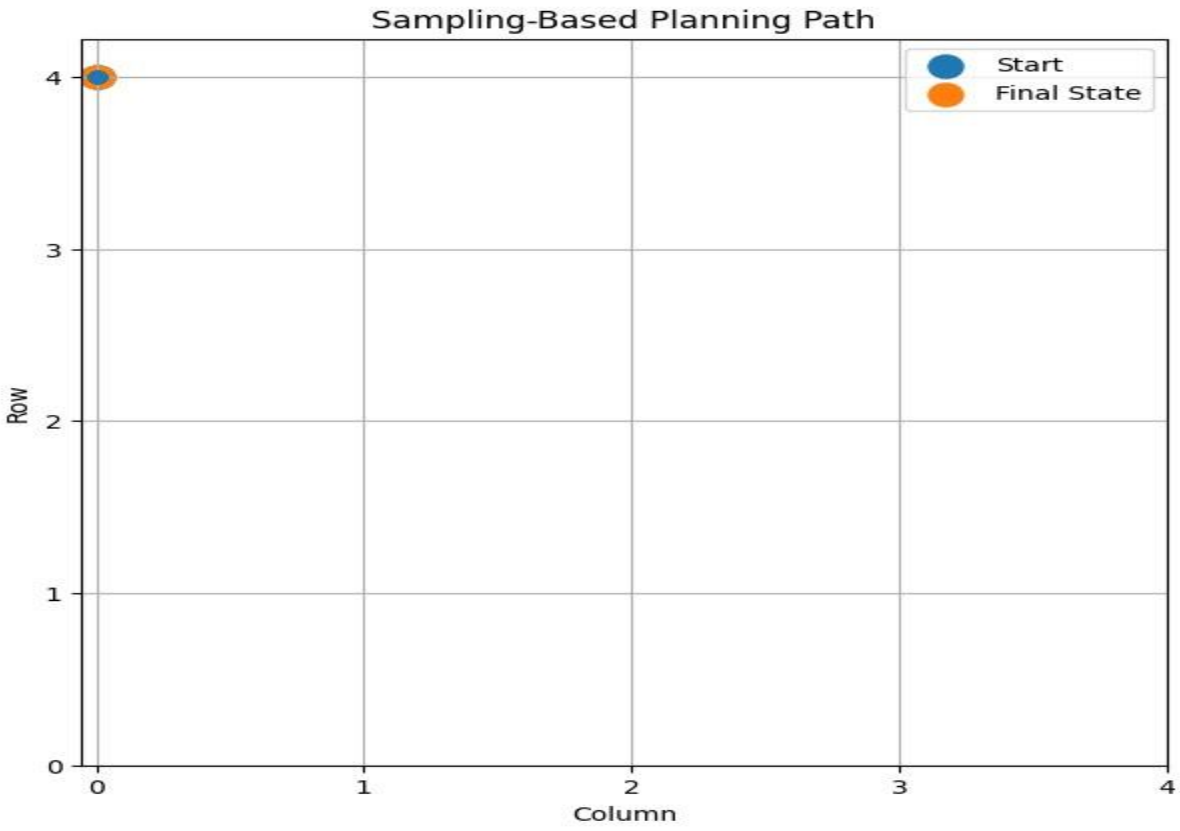
- 1000 experience samples are generated.
- A transition/reward model is learned from the samples.
- The planner estimates rewards for available actions and selects the best action.
- Performance graphs compare the sampling-based agent with a random agent.
- A path plot shows the agent's movement in GridWorld toward the goal.

11. OUTPUT

```
Generated samples: 1000 First
5 samples:
((4, 0), 'up', -1, (3, 0))
((0, 0), 'left', -1, (0, 0))
((1, 2), 'down', -1, (2, 2))
((0, 4), 'up', -1, (0, 4))
((4, 1), 'up', -1, (3, 1))
Learned model entries: 100
Starting State: (0, 0)
Action Scores: up
: -5.00 down   : -
5.00 left   : -5.00
right  : -5.00
Selected Best Action: up
Average Sampling-Based Reward: -30.0
Average Random Agent Reward: -27.1
```


Performance Comparison
Average Total Reward
0
−5
−10
−15
−20
−25
−30
Random Agent
Sampling-Based Agent


Episode-wise Performance
Total Reward
Episode
Random Agent
Sampling-Based Agent
0
−5
−10
−15
−20
−25
−30
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5



Best Path: [(0, 0), (0, 0)]
Final Reward: -30 Goal

Reached: False

11. CONCLUSION

The developed system demonstrates Model-Based Reinforcement Learning using data generation, model learning and sampling-based planning. The agent can evaluate possible actions before acting and provides measurable performance and path visualizations.